🎓 CAPSTONE PROJECT · 2026

AI-Based Phishing Detection System

Complete Project Documentation

Author: Nishanth G E | BE CSE (CyberSecurity), 3rd Year

Version: 2.0.0 | **Date:** March 2026

Tech: Node.js · Express.js · Naive Bayes · Random Forest · JWT

📚 AI-Based Phishing Detection System — Complete Project Documentation

Author: *Nishanth G E — BE CSE (CyberSecurity), 3rd Year*

Project Type: *Cybersecurity Capstone Project — 2026*

Version: *4.0.0*

Table of Contents

1. Project Overview
2. Technology Stack
3. Project Structure
4. AI / ML Engine
5. Database — MongoDB
6. How to Use (Setup & Run)
7. Authentication
8. Backend API Endpoints
   - Auth Endpoints
   - Email Analysis Endpoints
   - URL Analysis Endpoints
   - Data & Analytics Endpoints
   - Download Endpoints
   - System Endpoints
9. Frontend Features & Pages
1. Classification Thresholds
2. Security Features
3. Sample Test Cases
4. Datasets
5. Known Limitations & Disclaimer

---

1. Project Overview

The **AI-Based Phishing Detection System** is a full-stack cybersecurity application that uses **real machine learning** (not just keyword lists) to detect:

- **Phishing Emails** — Analyzed using a **Multinomial Naive Bayes** classifier trained on 160 labeled emails.
- **Malicious URLs** — Analyzed using a **Random Forest Ensemble** of 5 trees with weighted voting.

The system provides:

- A premium **dark cyberpunk / glassmorphism UI** dashboard at `http://localhost:8080`
- A fully documented **REST API** at `http://localhost:8081`
- Real-time threat scoring (0–100), explainable AI output, and JWT-authenticated sessions
- **MongoDB** for persistent storage of user accounts and analysis history
- Batch analysis, downloadable reports (JSON/CSV), and live analytics dashboard

---

2. Technology Stack

Backend

Component	Technology	Version
Runtime	**Node.js**	≥ 14.0.0
Web Framework	**Express.js**	^4.18.2
Database ODM	**Mongoose**	^8.x
ML / NLP Library	**natural** (NLP)	^6.5.0
Authentication	**jsonwebtoken (JWT)**	^9.0.3
Password Hashing	**bcryptjs**	^3.0.3
Security Headers	**helmet**	^7.1.0
CORS	**cors**	^2.8.5
Rate Limiting	**express-rate-limit**	^7.1.5
Request Logging	**morgan**	^1.10.0
URL Parsing	**url-parse**	^1.5.10
Input Validation	**validator**	^13.11.0
Dev Server Reload	**nodemon**	^3.0.2

Frontend

Component	Technology
Markup	HTML5 (Semantic)
Styling	Vanilla CSS with Glassmorphism + CSS variables
Scripting	Vanilla JavaScript (ES6+)
Charts	Chart.js (bundled locally)
Typography	JetBrains Mono + Inter (Google Fonts)
Theme	Dark cyberpunk with neon cyan/purple palette

Database

Store	Type	Collection
User accounts	**MongoDB** (Mongoose)	`users`
Email analysis history	**MongoDB** (Mongoose)	`emailanalyses`
URL analysis history	**MongoDB** (Mongoose)	`urlanalyses`

> ***Fallback:*** *If MongoDB is not running, the system automatically switches to in-memory storage so the server never crashes.*

---

3. Project Structure

```
phishing-ai-system/
│
├── src/
│   ├── backend/
│   │   ├── server.js                  # ★ Main Express server (port 8081)
│   │   ├── frontend-server.js         # Static file server (port 8080)
│   │   │
│   │   ├── config/
│   │   │   └── db.js                  # ★ MongoDB connection + fallback logic
│   │   │
│   │   ├── models/                    # ★ Mongoose Schemas
│   │   │   ├── User.js                # User schema (username, password)
│   │   │   ├── EmailAnalysis.js       # EmailAnalysis schema
│   │   │   └── URLAnalysis.js         # URLAnalysis schema
│   │   │
│   │   ├── routes/
│   │   │   ├── authRoutes.js          # Signup, login, profile, users
│   │   │   ├── emailRoutes.js         # Email analysis + download endpoints
│   │   │   ├── urlRoutes.js           # URL analysis + download endpoints
```

```
|   |   |   ├── dataRoutes.js           # Stored analyses retrieval
|   |   |   └── downloadRoutes.js       # Bulk JSON/CSV download routes
|   |   |
|   |   ├── controllers/
|   |   |   ├── authController.js       # Signup/login/JWT logic
|   |   |   ├── emailController.js      # Email analysis controller
|   |   |   └── urlController.js        # URL analysis controller
|   |   |
|   |   ├── services/
|   |   |   ├── emailAnalyzer.js        # Email ML classification service
|   |   |   └── urlAnalyzer.js          # URL ML classification service
|   |   |
|   |   ├── utils/
|   |   |   ├── aiEngine.js             # ★ Core ML engine (Naive Bayes + Random For
est)
|   |   |   ├── dataStore.js            # MongoDB-backed analysis store (async)
|   |   |   ├── userStore.js            # MongoDB-backed user store (async)
|   |   |   ├── explainableAI.js        # Human-readable explanation generator
|   |   |   └── featureExtractor.js     # URL feature extraction
|   |   |
```

```
│   │   └── store/
│   │       └── dataStore.js            # Store module used by routes/downloads
│   │
│   └── frontend/
│       ├── index.html                  # ★ Main dashboard UI
│       ├── script.js                   # Dashboard logic & API calls
│       ├── style.css                   # Glassmorphism dark theme
│       └── api-docs.html               # In-browser API documentation UI
│
├── datasets/
│   ├── phishing_keywords.json          # Reference keyword patterns
│   └── malicious_url_patterns.json     # Reference URL patterns
│
├── .env.example                        # ★ Environment variable template
├── package.json                        # NPM dependencies & scripts
├── generate_pdf.js                     # PDF generator script
├── start-both-servers.bat              # ★ One-click start script (Windows)
└── README.md                           # Quick start readme
```

4. AI / ML Engine

The core intelligence lives in `src/backend/utils/aiEngine.js`.

4.1 Email — Multinomial Naive Bayes

Trained at server startup on **160 labeled emails** (60 phishing + 100 legitimate):

Feature	Detail
Tokenization	Word-level, lowercase normalized
Stopword removal	37+ stopwords removed to reduce noise
Token deduplication	Per-document — repeated words counted only once
Formal email context	15 regex patterns detect cover letters, academic emails; dampens score by up to **85%**
Laplace smoothing	Prevents zero-probability failures on unknown tokens
Prior bias	55% legitimate class prior — reduces false positives

Thresholds:

```
threatScore ≥ 72  →  Phishing
threatScore ≥ 48  →  Suspicious

threatScore < 48  →  Safe
```

4.2 URL — Random Forest Ensemble (5 Trees)

Tree	Feature Group	Weight
Tree 1	Domain structure (IP addresses, subdomain count, HTTPS check)	30%
Tree 2	URL length & special character density	20%
Tree 3	Brand impersonation + suspicious TLDs (.tk, .ml, .cf...)	25%
Tree 4	URL shorteners & open redirect parameters	15%
Tree 5	Punycode detection, path depth, non-standard ports	10%

5. Database — MongoDB

5.1 Connection

Configured in `src/backend/config/db.js`. Default connection string:

```
mongodb://localhost:27017/phishing-ai-system
```

Override using environment variable:

```
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/phishing-ai-system
```

The server connects to MongoDB **before accepting requests**. If the connection fails, it **automatically falls back to in-memory storage** — no crash.

5.2 Mongoose Models

User Model (`models/User.js`)

Field	Type	Notes
`username`	String	Unique, required, min 4 chars
`password`	String	bcrypt hashed, 12 salt rounds
`createdAt`	Date	Auto-set by Mongoose timestamps

EmailAnalysis Model (`models/EmailAnalysis.js`)

Field	Type	Notes
`subject`	String	Extracted from email content
`content`	String	Preview (first 200 chars)
`fullContent`	String	Full email text
`classification`	String	`Safe` / `Suspicious` / `Phishing`
`threatScore`	Number	0–100
`isPhishing`	Boolean	true if classification is Phishing
`confidence`	Mixed	Level + value object
`fullAnalysis`	Mixed	Complete ML analysis object
`createdAt`	Date	Auto-set timestamp

URLAnalysis Model (`models/URLAnalysis.js`)

Field	Type	Notes
`url`	String	The analyzed URL
`classification`	String	`Safe` / `Suspicious` / `Malicious`
`threatScore`	Number	0–100
`isMalicious`	Boolean	true if classification is Malicious
`confidence`	Mixed	Level + value object
`fullAnalysis`	Mixed	Complete ML analysis with tree votes
`createdAt`	Date	Auto-set timestamp

5.3 View Data in MongoDB Compass

1. Download MongoDB Compass
2. Connect to: `mongodb://localhost:27017`
3. Open database: `phishing-ai-system`
4. Browse collections: `users` , `emailanalyses` , `urlanalyses`

---

6. How to Use (Setup & Run)

Prerequisites

- **Node.js** v14 or higher — Download
- **MongoDB** — Download
- npm (comes with Node.js)
- Modern browser (Chrome, Edge, Firefox)

Installation

```
cd d:\CapstoneNew\Capstone\phishing-ai-system
npm install
```

Configure Environment (Optional)

```
copy .env.example .env

Edit .env with your MongoDB URI and secrets
```

Starting MongoDB (Windows — as Administrator)

```
net start MongoDB
```

> *If you see "Access Denied", open PowerShell as Administrator and retry.*

Starting the Servers

Option A — One-click (Windows):

```
Double-click: start-both-servers.bat
```

Option B — Manual (two terminals):

```
# Terminal 1 — Backend API (port 8081)
node src/backend/server.js

Terminal 2 — Frontend UI (port 8080)

node src/backend/frontend-server.js
```

Accessing the System

Service	URL
🖥️ Frontend Dashboard	http://localhost:8080
⚙️ Backend API	http://localhost:8081
💚 Health Check	http://localhost:8081/api/health
📚 API Docs (UI)	http://localhost:8081/api/docs
📊 Data Stats	http://localhost:8081/api/data/stats

Demo Login Credentials

Field	Value
Username	`admin`
Password	`password123`

---

7. Authentication

The system uses **JWT (JSON Web Tokens)** for protecting analysis endpoints.

- Tokens expire after **7 days**
- Include the token in all protected requests:

```
Authorization: Bearer <your-jwt-token>
```

- Tokens are obtained from `POST /api/auth/login` or `POST /api/auth/signup`
- Passwords are hashed with **bcrypt** (12 salt rounds)

---

8. Backend API Endpoints

Base URL: `http://localhost:8081`

All responses follow this envelope:

```
{
  "success": true,

  "data": { ... },

  "metadata": { "timestamp": "...", "version": "2.0.0" }

}
```

8.1 Auth Endpoints

`POST /api/auth/signup`

Register a new user account.

Request Body:

```
{ "username": "string (min 4 chars)", "password": "string (min 6 chars)" }
```

Success Response (201):

```
{ "success": true, "data": { "token": "<jwt>", "username": "newuser" } }
```

Error Codes: `VALIDATION_ERROR` (400), `USER_EXISTS` (409)

`POST /api/auth/login`

Login with credentials.

Request Body:

```
{ "username": "admin", "password": "password123" }
```

Success Response (200):

```
{ "success": true, "data": { "token": "<jwt>", "username": "admin" } }
```

`GET /api/auth/profile`

Get authenticated user's profile. **Requires Bearer token.**

`GET /api/auth/users`

List all registered users.

8.2 Email Analysis Endpoints

`POST /api/analyze-email` ⭐

Analyze a single email using Naive Bayes ML.

Request Body:

```
{ "emailContent": "URGENT: Your PayPal account has been suspended..." }
```

Response includes: `classification`, `threatScore`, `confidence`, `mlEngine`, `explanation`

`POST /api/analyze-emails-batch`

Analyze up to **10 emails** in one request.

Request Body:

```
{ "emails": [{ "emailContent": "..." }, { "emailContent": "..." }] }
```

`GET /api/email-stats`

Aggregated statistics for email analyses in this session.

`POST /api/validate-email`

Validate email content format before analysis.

`GET /api/sample-email`

Get a sample email for testing. Query: `?type=phishing|legitimate|suspicious`

8.3 URL Analysis Endpoints

`POST /api/analyze-url` ⭐

Analyze a single URL using Random Forest ML.

Request Body:

```
{ "url": "http://paypal-security.verify-account.tk/login/confirm" }
```

Response includes: `classification`, `threatScore`, `confidence`, `mlEngine.treeVotes`, `explanation`

`POST /api/analyze-urls-batch`

Analyze up to **20 URLs** in one request.

`GET /api/url-stats`

Aggregated statistics for URL analyses.

`POST /api/validate-url`

Validate URL format before analysis.

`GET /api/sample-url`

Get sample URLs. Query: `?type=malicious|suspicious|legitimate`

`POST /api/extract-url-components`

Extract/analyze URL structure without full ML scoring.

8.4 Data & Analytics Endpoints

Method	Endpoint	Description
`GET`	`/api/data/analyses`	All analyses. Query: `?limit=100`
`GET`	`/api/data/emails`	All email analyses. Query: `?limit=50`
`GET`	`/api/data/urls`	All URL analyses. Query: `?limit=50`
`GET`	`/api/data/email/:id`	Specific email analysis by MongoDB `_id`
`GET`	`/api/data/url/:id`	Specific URL analysis by MongoDB `_id`
`GET`	`/api/data/stats`	Overall statistics (totals, threat breakdown, storage type)
`DELETE`	`/api/data/clear`	Clear all stored analysis records

8.5 Download Endpoints

All under `/api/download/`:

Method	Endpoint	Description
`GET`	`/api/download/emails/json`	View email records as JSON
`GET`	`/api/download/emails/download-json`	Download emails as `.json` file
`GET`	`/api/download/emails/download-csv`	Download emails as `.csv` file
`GET`	`/api/download/urls/json`	View URL records as JSON
`GET`	`/api/download/urls/download-json`	Download URLs as `.json` file
`GET`	`/api/download/urls/download-csv`	Download URLs as `.csv` file
`DELETE`	`/api/download/clear`	Clear all stored data

CSV columns — Emails: `ID, Timestamp, Subject, Threat Score, Classification, Is Phishing` **CSV columns — URLs:** `ID, Timestamp, URL, Threat Score, Classification, Is Malicious`

8.6 System Endpoints

Method	Endpoint	Description	Auth
`GET`	`/api/health`	Health check with uptime, memory, version	No
`GET`	`/api/docs`	API documentation (HTML UI)	No
`GET`	`/api/docs/json`	API reference (JSON)	No

---

9. Frontend Features & Pages

Main Dashboard — `http://localhost:8080`

Section	Description
Login / Signup	JWT-based auth with demo credentials
Email Analyzer	Paste email text → instant ML result
URL Analyzer	Paste URL → threat score + tree-vote breakdown
Sample Data Buttons	Load threat or safe samples for quick testing
Results Panel	🟢 Safe / 🟡 Suspicious / 🔴 Phishing/Malicious
Analytics Chart	Live Chart.js threat distribution
Toast Notifications	Neon-glow alerts

Design System

- **Colors:** Neon cyan (`#00d4ff`) + purple (`#7c3aed`) on `#0a0a0f`
- **Glassmorphism:** `backdrop-filter: blur()` frosted panels
- **Typography:** `JetBrains Mono` for scores, `Inter` for text

---

10. Classification Thresholds

Threat Score	Email Label	URL Label	Color
0 – 47	✅ **Safe**	✅ **Safe**	Green
48 – 71	⚠️ **Suspicious**	⚠️ **Suspicious**	Yellow
72 – 100	🚨 **Phishing**	🚨 **Malicious**	Red

---

11. Security Features

Feature	Implementation	Details
HTTP Security Headers	**Helmet.js**	CSP, X-Frame-Options, HSTS
Cross-Origin Protection	**CORS**	Whitelisted origins only
Rate Limiting	**express-rate-limit**	100 requests / 15 min per IP
Authentication	**JWT**	Bearer token, 7-day expiry
Password Hashing	**bcryptjs**	12 salt rounds
Timing-Safe Login	**crypto.timingSafeEqual**	Prevents timing attacks
Input Validation	**validator.js**	Format and size checks
Payload Limit	**Express body parser**	Max 10MB per request
NB Edge Cases	**Laplace Smoothing**	Prevents zero-probability issues

---

12. Sample Test Cases

Phishing Email (Expected: Score ~95–100)

```
URGENT: Your PayPal account has been suspended.
Click here immediately to verify your identity or your account

will be permanently deleted. Provide your SSN and credit card now.
```

Legitimate Email (Expected: Score < 48)

```
Dear Team, Please find attached the quarterly report for your review.
Let me know if you have any questions.

Best regards, John Smith
```

Malicious URL (Expected: Score ~85+)

```
http://paypal-security.verify-account.tk/login/confirm
```

Safe URL (Expected: Score < 48)

```
https://www.google.com/search?q=cybersecurity
```

---

13. Datasets

Located in `datasets/`:

File	Description
`phishing_keywords.json`	Reference phishing keyword patterns
`malicious_url_patterns.json`	Reference malicious URL patterns

> *The **Naive Bayes** classifier trains on 160 inline email examples in `aiEngine.js` — no external dataset file required.*

---

14. Known Limitations & Disclaimer

> ⚠️ ***This system is built for educational and demonstration purposes only.***

Limitation	Detail
Fallback mode	If MongoDB is offline, data resets on server restart
Small training set	NB trained on 160 emails — may have edge-case errors
No real-time threat feeds	Heuristic-based, no live CVE/threat database
No HTTPS	Runs on plain HTTP locally
Single machine	No clustering or load balancing

Generated: March 2026 | AI-Based Phishing Detection System v4.0.0